

Zeno Oracle: A Proof-Carrying Oracle Architecture for Fail-Closed DeFi Consumers

Dana Edwards

Devnet Alpha Whitepaper



Abstract

Zeno Oracle is a proof-carrying oracle layer for ZenoDEX-critical reads. Its core design principle is that a consumer should not accept an oracle value merely because a service returned it. A consumer should accept a value only when a replayable artifact proves that the value passed the declared feed, reporter, source-diversity, aggregation, freshness, uncertainty, and consumer-profile checks. The current devnet alpha implements reporter registration, feed registration, signed report submission, admitted median aggregation, read receipt generation, ZenoDEX adapter bridge generation, economic event receipts, and replay from a persisted event log. This paper gives the mathematical contract, verifier design, current evidence boundary, and remaining production work.

1 Motivation

Oracle failures are not only wrong-price failures. They also include stale reports, unverifiable reports, source correlation, policy downgrades, receipt borrowing across consumers, replay drift, unbounded uncertainty, reward overspending, and consumer-side bypasses. A conventional API-oriented oracle integration can accidentally turn these into hidden assumptions.

Zeno Oracle exposes those assumptions as explicit artifacts. The target safety shape is

$$\text{AcceptedOracleUse} \Rightarrow \text{DeclaredOraclePolicySatisfied}.$$

This does not prove that a reported price is objectively true. It proves that a critical consumer only used an oracle value after the declared safety contract was checked.

2 System Model

Let $C(x)$ denote canonical JSON serialization of an artifact x , and let

$$H(x) := \text{sha256} : \parallel \text{SHA256}(C(x)).$$

Most Zeno Oracle objects carry an identifier field such as `feed_id`, `submission_id`, `aggregate_id`, or `bridge_id`. For an object x with identifier field id , the verifier checks

$$x.id = H(x \setminus \{id\}).$$

Any mutation to a serialized semantic field changes the expected identifier and causes verifier rejection.

The devnet service stores accepted artifacts under hash-derived filenames and appends event receipts to `events.jsonl`. A live state is intended to be replayable from

event log + referenced artifacts.

Replay currently rejects when a referenced artifact is missing. A production replay kernel should also re-run every referenced verifier and compare all artifact hashes.

3 Feed Registry

A feed registry is a versioned set of active feeds. Each feed contains a query specification, a source-diversity policy, and an aggregate policy. For the current price-feed shell, a query specification is restricted to

```
query_kind = price_e8,
            unit = quote_per_base,
value_scale = 108,
base_asset ≠ quote_asset.
```

The aggregate policy declares the aggregation schema, report schema, evidence floor, reporter quorum, source quorum, freshness window, and maximum deviation in basis points. A feed is rejected when any hash, active-status, evidence-floor, quorum, freshness, deviation, or uniqueness check fails.

4 Source Diversity

For a source set S , the verifier counts distinct operators, venues, data families, transports, and jurisdictions. It also checks concentration limits:

$$\begin{aligned} \text{DistinctOperators}(S) &\geq m_o, \\ \text{DistinctVenues}(S) &\geq m_v, \\ \text{DistinctFamilies}(S) &\geq m_f, \\ \text{DistinctTransports}(S) &\geq m_t, \\ \text{DistinctJurisdictions}(S) &\geq m_j, \\ \text{MaxSameClass}(S) &\leq M. \end{aligned}$$

This proves compliance with the declared classification rule. It does not prove real-world independence or source honesty.

5 Signed Reports

A signed report submission contains a reporter id, reporter BLS public key, chain id, and one or more reports. Each report contains a query id, source id, value, observed epoch, sequence, previous-report id, payload hash, signature, and report id.

The signed payload is the canonical serialization of the domain-separated report fields. The verifier checks

$$\begin{aligned} \text{payload_hash} &= H(\text{payload}), \\ \text{Verify}_{BLS}(\text{pubkey}, \text{domain} \parallel \text{payload}, \text{signature}) &= \text{true}, \\ \text{report_id} &= H(\text{report} \setminus \{\text{report_id}\}). \end{aligned}$$

It also checks sequence-chain continuity and previous-report links. Signature acceptance proves key possession over the exact payload. It does not prove reporter honesty.

6 Reporter Lifecycle and Admission

Reporter lifecycle traces model registration, bond deposits, report submission, disputes, slashes, dispute resolution, unregistration, and bond withdrawal. A reporter is admissible only when the lifecycle trace proves the reporter is active and sufficiently bonded at submission time.

For a signed report r , lifecycle trace L , source policy S , current epoch t , and freshness window w , the admission predicate is

$$\begin{aligned} \text{Admit}(r, L, S, t, w) &:= \text{SignedReportAccepted}(r) \\ &\quad \wedge \text{LifecycleSubmitMatches}(L, r) \\ &\quad \wedge \text{ReporterActiveAndBonded}(L, r.\text{reporter_id}) \\ &\quad \wedge r.\text{source_id} \in S \\ &\quad \wedge r.\text{query_id} = S.\text{query_id} \\ &\quad \wedge r.\text{observed_epoch} \leq t \\ &\quad \wedge t - r.\text{observed_epoch} \leq w. \end{aligned}$$

The devnet service uses this bridge when accepting signed report submissions. Signed but unregistered, stale, future, wrong-query, wrong-source, or unbonded reports are rejected before they can enter an aggregate.

7 Median Aggregation and Uncertainty

The current admitted aggregate is `median_3`: exactly three admitted reports from distinct reporters and distinct sources for the same query. Given admitted values $x_1, x_2, x_3 > 0$, the aggregate value is

$$m := \text{median}(x_1, x_2, x_3).$$

The confidence radius is

$$c := \max_i |x_i - m|,$$

and the uncertainty in basis points is

$$\text{deviation_bps} := \left\lceil \frac{10000c}{m} \right\rceil.$$

The aggregate is accepted only when

$$\text{deviation_bps} \leq \text{max_deviation_bps}.$$

A price is high uncertainty when the reported spread around the median exceeds the feed’s declared deviation budget, or when a consumer profile requires a tighter uncertainty budget than the aggregate supplies. For critical actions, high uncertainty is a rejection condition, not a warning.

8 Read Receipts and Consumer Binding

An aggregate read bridge converts an accepted aggregate into a consumer-readable receipt bundle. The value hash commits to the aggregate id, query id, value, confidence radius, deviation, observed epoch, report count, and admission count. The bridge then creates a read receipt and a consumer-action receipt that depends on the read receipt.

A ZenoDEX consumer should not merely ask whether a read exists. It should ask whether the read is authorized for this exact action. The aggregate-adapter bridge binds

$$\text{aggregate_read}, \quad \text{action}, \quad \text{consumer_profile}.$$

For critical actions, the profile pins the consumer module, action kind, query id, evidence floor, maximum freshness window, and criticality bit. This closes the receipt-borrowing shape: a valid read for one query, module, action, value, epoch, or profile cannot be silently reused for a different critical action.

9 Devnet Service

The devnet service is a transport layer around verifier kernels:

$$\text{HTTPAccepted} \Rightarrow \text{VerifierAccepted} \Rightarrow \text{ArtifactPersisted} \Rightarrow \text{EventReceiptAppended}.$$

It exposes:

- GET /health
- GET /state
- POST /reporters/register
- POST /feeds/register
- POST /reports/submit
- POST /aggregates/build
- GET /reads/latest?query_id=...
- GET /adapter/latest?query_id=...
- POST /economics/event
- POST /replay

The event append path is serialized by a service lock so concurrent requests inside one process cannot assign the same event sequence.

10 Economic Security Envelope

The current economic security shell verifies local budget arithmetic rather than a live token economy. It enforces

$$\begin{aligned} \text{query_reward_paid} &\leq \text{query_budget_remaining}, \\ \text{reporter_slash_paid} &\leq \text{reporter_bond_available}, \\ \text{dispute_slash_paid} &\leq \text{dispute_bond_available}, \\ \text{reporter_fee_share} + \text{treasury_fee_share} + \text{burn_fee_share} &\leq \text{fee_paid}. \end{aligned}$$

This is the fail-closed budget layer. It does not yet prove optimal tokenomics, oracle-token valuation, or game-theoretic equilibrium.

11 Current Safety Statements

The current implementation supports the following verifier-backed statements:

1. **Content mutation rejection.** If an accepted object uses a content-addressed identifier, then changing any serialized semantic field without recomputing the identifier causes verifier rejection.
2. **Signed payload binding.** If a signed report submission is accepted, every accepted report signature verifies over the exact canonical payload fields used to compute the payload hash.
3. **Admission safety.** If a report admission is accepted, then the signed report, lifecycle submit event, source-diversity policy, query id, payload hash, and freshness window are mutually consistent.
4. **Median correctness.** If an admitted median3 aggregate is accepted, then its value, confidence, deviation, observed epoch, reporter count, source count, and deviation-policy check are derived from exactly three admitted reports.
5. **Consumer binding.** If an aggregate-adapter bridge is accepted, then the downstream action facts match the accepted read bundle and are no weaker than the declared consumer profile.
6. **Budget non-overspend.** If a budget transition is accepted, then reward, slash, dispute, and fee-split spends do not exceed their declared envelopes.
7. **Devnet replay reference integrity.** If devnet replay accepts, every event-log artifact reference observed by the replay command resolves to a file in the devnet store.

12 Evidence Frontier

As of the devnet-alpha gate, the public replay surface includes:

$$\begin{aligned} \text{oracle_chaos_case_count} &= 283, \\ \text{oracle_chaos_rejected_count} &= 283, \\ \text{oracle_chaos_failed_count} &= 0, \\ \text{oracle_pytest_count} &= 275, \\ \text{devnet_alpha_criteria} &= 12/12. \end{aligned}$$

The devnet-alpha audit exercises local service startup, three reporter registrations, three signed report submissions over HTTP, feed registry persistence, admitted median aggregate construction, read API exposure, ZenoDEX adapter bridge generation, economic reward event persistence, replay from stored receipts, CI wiring, public documentation, and devnet package receipt generation.

13 Limitations

The devnet alpha does not claim production oracle truth, public on-chain feed governance, a production oracle token, production code signing, real-world source honesty, complete Byzantine fault tolerance, final tokenomics, or full replay equivalence under arbitrary crash-consistency failures.

14 Production Roadmap

The next technical milestones are:

1. Stateful disaster-sequence fuzzing for the HTTP service and replay store.
2. Replay strengthening: re-verify every referenced artifact and compare live state to replayed state.
3. Higher-redundancy aggregation beyond `median_3`.
4. Public feed-governance rules for source classification, feed upgrades, and policy downgrades.
5. Production reporter economics: bond sizing, dispute incentives, reward budgets, slash rules, and concentration caps.
6. Production code signing and platform-native binaries.
7. Lean or SMT proofs for median/deviation arithmetic, budget non-overspend, and replay state equivalence.

15 Related Zeno Oracle Specifications

- `docs/ZENO_ORACLE_DEVNET_ALPHA.md`
- `docs/ZENO_ORACLE_MVP_STATUS.md`
- `docs/ZENO_ORACLE_FEED_REGISTRY_V1.md`
- `docs/ZENO_ORACLE_SIGNED_REPORT_V1.md`
- `docs/ZENO_ORACLE_REPORT_ADMISSION_V1.md`
- `docs/ZENO_ORACLE_ADMITTED_MEDIAN3_V1.md`
- `docs/ZENO_ORACLE_AGGREGATE_READ_V1.md`
- `docs/ZENO_ORACLE_AGGREGATE_ADAPTER_V1.md`
- `docs/ZENO_ORACLE_ECONOMIC_SECURITY_V1.md`
- `docs/ZENO_ORACLE_CHAOS_ENGINEERING.md`
- `docs/ZENO_ORACLE_PRODUCTION_GATES.md`